

Edge Language Coach — Architecture Report

Scalable & Reliable Services

Contents

1. System Overview	1
2. Reliability	2
2.1 Circuit breaker — measured behaviour	2
2.2 Other primitives (summary)	3
2.3 Tests in CI	3
3. Scalability	3
3.1 Mechanism	4
3.2 Measured scale-out behaviour	4
3.3 Threats to validity	4
3.4 Production deployment	5
4. Observability	5
4.1 Structured logging	5
4.2 Prometheus metrics	5
4.3 Dashboards & probes	6
5. Operator Surface — Agentic MCP	6
6. Trade-offs	7
7. Future Work	7

1. System Overview

Edge Language Coach is a small web app for practising Italian. It lets you chat with an LLM tutor, scrapes a few RSS feeds in the background to keep the conversation topics fresh, and turns each session into spaced-repetition flashcards afterwards.

We built it as three independently runnable services behind a load balancer, with Supabase for the database (and auth) and Groq for the actual model calls.

Unit	Technology	Role
Web	React 19 + Vite, served by nginx	Single-page app, static build
Gateway	Fastify 5 (Node.js 22)	REST API, auth, reliability primitives
Workers	Node.js + BullMQ	Async jobs (scraper, summary, flashcard)
Redis	redis:7-alpine	BullMQ queues + distributed rate-limit counters
Observability	Prometheus + Grafana	Metrics scraping and dashboards

The gateway holds no session state in memory. Every request carries a Supabase JWT, and every rate-limit counter lives in Redis. That’s the property that makes horizontal scale-out a one-line change later (Section 3).

The request hot path:

`Browser → web nginx (:80) → nginx-lb (:3001) → gateway:N → Redis | Supabase | Groq`

The async path (long-running work):

`gateway → Redis (BullMQ queue) → workers → Supabase`

Anything that takes more than ~2 seconds of LLM time goes onto a queue: post-session summaries, flashcard pack generation, the periodic topic scrape.

2. Reliability

We built five reliability primitives, each pointed at a specific failure mode we could see coming.

Primitive	Code location	Failure addressed
Circuit breaker (opossum) on Groq	<code>apps/gateway/src/plugins/groq.ts</code>	Groq API slowdown / outage
Per-user rate limit (Redis-backed)	<code>apps/gateway/src/server.ts</code>	Abusive client flooding the gateway
BullMQ retry + exponential backoff	<code>apps/workers/src/lib/queues.ts</code>	Transient worker / downstream failure
Dead-letter logging	<code>apps/workers/src/index.ts</code>	A job failing past its retry budget
Scraper idempotency key in Redis	<code>apps/workers/src/jobs/scraper.job.ts</code>	Duplicate work after a worker restart

2.1 Circuit breaker — measured behaviour

To check the breaker actually does what it’s supposed to, we wrote a small mock Groq server (`mocks/groq-mock.mjs`) that we can put into “slow” mode (30 s sleep on every request, well past the 15 s breaker timeout). Then a driver script (`mocks/breaker-driver.mjs`) fired 60 requests

at concurrency 12 against the gateway. The full output is at `load/breaker-demo-output.txt`. Summary:

Phase	Calls	Per-call latency	Status
Closed (probing)	1 – 7	15.3 s (timing out against slow upstream)	breaker collecting failure samples
Transition	call 7	—	volumeThreshold (10) + 50 % error rate crossed → opened
Open (short-circuit)	8 – 60	12 – 58 ms	requests fail-fast, never reach upstream

The headline number is the latency cliff: 15,300 ms → 25 ms between adjacent calls. Without the breaker, all 60 requests would have queued up against the slow upstream and paid the full 15 s timeout each. With it, only 7 calls actually paid that cost — the other 53 cost effectively zero CPU and didn’t even open a socket to Groq.

The breaker state is also exposed as a Prometheus gauge (`groq_circuit_breaker_state`), so the closed → open → half-open transitions show up live on the Grafana dashboard while the demo is running.

2.2 Other primitives (summary)

- **Rate limit:** 60 requests/minute per authenticated user, 60/minute per IP otherwise. The counters live in Redis (`@fastify/rate-limit` Redis store), so the limit is shared across gateway replicas — no sticky sessions needed. Health probes are allow-listed.
- **Retries:** All BullMQ workers use `attempts: 3`, `backoff: { type: 'exponential', delay: 5000 }`. After three failed attempts the job is logged at `error` level with `dlq: true` and shows up in the Bull Board UI at `:3002/queues`.
- **Idempotency:** The scraper sets a Redis key `scraper:slot:<date>:<6h-window>` on success, with TTL = 6 h. A second invocation in the same window is a no-op, so if a worker crashes mid-scrape and the job is replayed, we don’t get duplicate topics.

2.3 Tests in CI

The reliability story above isn’t only asserted in prose — the gateway has a small Vitest suite (Fastify `inject()` harness, Redis / Supabase / Groq mocked at module boundaries) that pins the readiness contract: `/livez` returns 200, `/readyz` returns 200 when deps are up, and `/readyz` returns 503 with the failing dependency named in the body when Redis or Groq go down. CI runs `pnpm turbo test` ahead of typecheck and build on every push and PR, so a regression in the failure paths fails the pipeline. Source: [apps/gateway/src/__tests__/health.test.ts](#). Breaker, rate-limit, and queue-retry tests are the natural next additions.

3. Scalability

The gateway scales horizontally; workers and Redis are vertically bounded, but we made that choice on purpose (see Section 6).

3.1 Mechanism

```
docker compose up --scale gateway=3
```

An nginx load balancer (`nginx-lb`) sits in front of the gateway pool and round-robins traffic via Docker’s built-in DNS. The web SPA and Prometheus both target `nginx-lb:3001`, so adding replicas needs no client-side change and no scrape-config change.

3.2 Measured scale-out behaviour

The k6 script at `load/gateway.js` runs three scenarios back-to-back in one invocation:

Scenario	Profile
<code>baseline</code>	10 VUs constant for 30 s
<code>stressed</code>	Ramp 0 → 50 VUs over 60 s, then taper
<code>scaled_out</code>	50 VUs constant for 30 s

We ran the whole script twice — once with one gateway replica, once with three — keeping everything else fixed. Comparison:

Metric	1 replica	3 replicas	Δ
<code>gateway_errors</code> rate	6.81 %	0.27 %	−96 %
<code>scaled_out</code> p95 latency	2.43 s	2.15 s	−12 %
<code>stressed</code> p95 latency	1.73 s	1.84 s	+6 % (within noise)
Sustained throughput	39.3 req/s	39.4 req/s	unchanged

The error rate is where the scaling actually shows up. With one replica, the gateway was dropping ~7 % of requests under saturation; with three, it dropped to under 0.3 %. Latency under saturation only improved a little (~12 %), and throughput was basically flat.

What we take from this: under sustained load the **gateway itself** is the availability bottleneck — a single Node event loop can’t keep up with 50 concurrent VUs once you add auth verification and rate-limit checks on top, so it starts dropping connections. The **downstream hot path** (Supabase RTT, Redis round-trips) is the latency bottleneck, and adding gateway replicas doesn’t help with that, because the per-request work outside the gateway hasn’t changed.

There’s also a persistent ~25 % `http_req_failed` rate in both runs. That’s not a system failure — it’s the rate-limiter doing its job. 50 VUs at ~10 req/s is roughly 500 req/min, well over the configured 60 req/min cap, so most of those “failures” are 429s.

3.3 Threats to validity

A few things worth being honest about:

- **Co-located load generator.** k6 ran on the same host as Docker Desktop, competing for CPU. The clearest sign of this is the unexpected `baseline` p95 regression on the 3-replica run.

- **Mocked Groq.** All Groq calls returned in under 1 ms. That’s deliberate — we wanted to isolate the gateway/queue layer — but it means the absolute p95 numbers aren’t what you’d see in production.
- **One iteration per configuration.** No statistical variance, no confidence intervals.

3.4 Production deployment

The same compose model that demonstrates scale-out locally also runs in production, on a single Oracle Cloud VM via [docker-compose.prod.yml](#). The prod compose pulls multi-arch images (linux/amd64 + linux/arm64) from GHCR rather than building on the host, so deployment is image-pull + container-restart rather than a source build.

Three sequential GitHub Actions jobs ([.github/workflows/ci.yml](#)) run on every push to main:

1. **typecheck-build** — `pnpm install`, `prisma generate`, `pnpm turbo test`, then typecheck and build across all workspaces.
2. **docker** — `docker buildx` builds and pushes multi-arch images for `gateway` and `workers` to GHCR with a GHA build cache.
3. **deploy** — SSH into the Oracle host, `git pull`, `docker login ghcr.io`, `docker compose -f docker-compose.prod.yml pull && up -d --remove-orphans`, prune dangling images.

The prod compose intentionally omits Prometheus / Grafana / Bull Board (separate observability deploy) and the SPA (served from Cloudflare Pages / Vercel using the same `apps/web` build).

4. Observability

We went with metrics + structured logs and skipped distributed tracing. The trade-off is documented in Section 6; it was a scope call, not an oversight.

4.1 Structured logging

All services emit single-line JSON via `pino`. Every log line carries `service` (`gateway` / `workers`), a `requestId` (taken from an `X-Request-Id` header if present, generated otherwise), and the relevant domain identifiers (`sessionId`, `jobId`, `queue`). Log level is tunable via `LOG_LEVEL`; the default in production is `info`.

4.2 Prometheus metrics

Both the gateway and the workers expose `/metrics` in Prometheus text format, scraped every 15 s. The custom metrics that matter:

Metric	Type	Labels	Where
<code>http_request_duration_seconds</code>	Histogram	method, route, status_code	gateway
<code>groq_request_duration_seconds</code>	Histogram	operation, status	gateway
<code>groq_circuit_breaker_state</code>	Gauge	breaker	gateway
<code>bullmq_job_duration_seconds</code>	Histogram	queue, status	workers
<code>jobs_dead_letter_total</code>	Counter	queue	workers
<code>queue_depth_total</code>	Gauge	queue	gateway, workers

Metric	Type	Labels	Where
<code>active_sessions_total</code>	Gauge	—	gateway

Node default metrics (event-loop lag, GC, RSS) are also collected via `collectDefaultMetrics`.

4.3 Dashboards & probes

A Grafana dashboard provisioned from `docker/grafana/dashboards/edge-coach.json` is auto-loaded on startup at `localhost:3000`. Panels: request rate by route, p50/p95/p99 latency, 5xx rate, circuit-breaker state, queue depth, active sessions.

Two health endpoints back container orchestration:

- `GET /livez` — always returns 200, just a liveness signal.
- `GET /readyz` — pings Redis, Supabase, and Groq with 2 s timeouts; returns 503 with the failing dependency named in the body.

5. Operator Surface — Agentic MCP

On top of the observability stack we added an operator surface: two small [Model Context Protocol](#) servers that let an agent (Claude Desktop, Claude Code, or any other MCP client) inspect and remediate the running system without anyone having to SSH in or redeploy. The idea is to close the loop on top of what Prometheus already gives us — **detect** (Grafana / alerts) → **diagnose** (observability-mcp) → **remediate** (remediation-mcp).

We split the two servers by privilege on purpose:

Server	Surface	Tools	Auth
observability-mcp	Read-only over <code>/metrics</code> and <code>/readyz</code>	<code>get_service_health</code> , <code>get_queue_metrics</code> , <code>get_circuit_breaker_state</code> , <code>get_active_sessions</code> , <code>get_groq_latency</code>	None (stdio, locally scoped)
remediation-mcp	Guarded mutations on Redis + gateway admin	<code>pause_queue</code> , <code>resume_queue</code> , <code>reset_circuit_breaker</code> , <code>flush_dead_letter_queue</code>	Layered: <code>zod.enum</code> queue allow-list, <code>ADMIN_API_KEY</code> for the breaker reset, <code>confirm: true</code> for the irreversible flush, structured stderr audit log on every call

In practice: when the breaker demo trips the chat breaker, recovering becomes a three-step dialogue with the agent — call `get_circuit_breaker_state` to confirm the breaker is open, check that Groq is actually healthy again, then call `reset_circuit_breaker` to force-close it. Every action leaves an audit line on stderr (`stdout` is reserved for the MCP transport).

This doesn't replace distributed tracing — there are still no cross-service spans — but it does give us a typed way to ask the live system what's happening, which is the main thing we'd otherwise be reaching for tracing to do.

Sources: [apps/observability-mcp/src/index.ts](#) · [apps/remediation-mcp/src/index.ts](#) · [apps/gateway/src/routes/admin.ts](#).

6. Trade-offs

Four trade-offs worth calling out (full reasoning is in the ADRs in the repo):

Decision	Alternative	Why we chose this
Node.js workers	Go microservice	Same TypeScript types as the gateway, same toolchain, faster to iterate on
BullMQ + Redis	RabbitMQ / Kafka	One Redis instance does both queueing and rate-limiting, which keeps the moving parts down for a course-sized project
Supabase	Self-hosted Postgres + bespoke auth	Saved us weeks on auth; the cost is an external dependency we can't take fully offline
<code>prom-client</code> only	OpenTelemetry traces	Faster to instrument, fewer services to run; the cost is no cross-service tracing — we accepted this because the call graph is shallow (gateway → Groq, gateway → workers via queue)

The biggest one is the last. A more thorough submission would include OTel traces showing a request span propagated from gateway → BullMQ job → Groq. We've put it in future work; the MCP operator surface in Section 5 closes part of the gap by making live state queryable, but it's not a real substitute.

7. Future Work

- **OpenTelemetry traces.** Auto-instrument Fastify + ioredis + Prisma; carry trace context through the BullMQ job payload so the async path ends up in the same trace.
- **Worker autoscaling.** Workers are a fixed count right now. BullMQ already supports multiple consumers, so this is mostly wiring + manifests rather than a redesign.
- **Kubernetes manifests + HPA.** The compose setup is the demo target. Production would lift this into a Helm chart with a horizontal pod autoscaler driven by `queue_depth_total` and `http_request_duration_seconds`.
- **Statistical load-testing.** Multiple iterations per configuration with confidence intervals, run from a dedicated load-generation host (not co-located with Docker).

- **Tighten remediation-mcp auth.** Right now `pause_queue` / `resume_queue` rely on `stdio-locality` and the `zod.enum` queue allow-list. In production we'd require `ADMIN_API_KEY` for everything that mutates state, not just the breaker reset.

See the repository for the full ADRs (`docs/adr/`), the Mermaid C4 diagrams (`docs/c4-diagram.md`), the SLO table (`docs/slo-table.md`), and the raw k6 / breaker outputs (`load/`).